

Overview

In this lecture note, we will mostly go over the derivative of softmax function, which appeared in “Linear Models for Classification” and “Neural Networks” topic.

1.1 Softmax function

By definition, a softmax function is expressed as:

$$f(a_k) = \frac{\exp(a_k)}{\sum_j^M \exp(a_j)}, \quad (1)$$

where $a_j \in \mathbb{R} \ \forall j = 0, \dots, M$.

This function helps us interpret the output values in a probabilistic manner, because $\sum_j f(a_j) = 1$.

1.2 Softmax function is multi-class logistic regression

With K different classes in a classification problem, we use 1-of- K coding scheme, where $\mathbf{t} = (0, \dots, 1, \dots, 0)^T$ such that at k -position it has a value of 1 and the rest of the values are 0 to represent that $\mathbf{t} \in \mathcal{C}_k$. Alternatively, $t_{nk} = 1$. For instance, if we have $K = 5$, and if any n -th data point belongs to $\mathcal{C}_4$, $\mathbf{t}_n = (0, 0, 0, 1, 0)^T$.

In this case, if we are learning a classifier $y_k(\mathbf{x}_n, \mathbf{w})$ to predict the probability of $p(\mathcal{C}_k|\mathbf{x}_n)$, using Bayes's theorem, we are essentially looking for

$$p(\mathcal{C}_k|\mathbf{x}_n) = y_k(\mathbf{x}_n, \mathbf{w}) \quad (2)$$

$$= \frac{p(\mathbf{x}_n|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x}_n|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}_n|\mathcal{C}_k)p(\mathcal{C}_2) + \dots + p(\mathbf{x}_n|\mathcal{C}_K)p(\mathcal{C}_K)} \quad (3)$$

$$= \frac{p(\mathbf{x}_n|\mathcal{C}_k)p(\mathcal{C}_k)}{\sum_j p(\mathbf{x}_n|\mathcal{C}_j)p(\mathcal{C}_j)} \quad (4)$$

This equation can be transformed to a softmax function, if we replace $p(\mathbf{x}_n|\mathcal{C}_k)p(\mathcal{C}_k)$ with $a_k = \ln(p(\mathbf{x}_n|\mathcal{C}_k)p(\mathcal{C}_k))$ resulting into

$$p(\mathcal{C}_k|\mathbf{x}_n) = y_k(\mathbf{x}_n, \mathbf{w}) = \frac{\exp(a_k)}{\sum_j^M \exp(a_j)}. \quad (5)$$

1.3 Gaussian Discriminant Analysis

We can choose a relatively difficult route to approximate $p(\mathcal{C}_k|\mathbf{x}_n)$ by approximating the parameters of the Gaussian distribution of

$$p(\mathbf{x}_n|\mathcal{C}_k) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\mathbf{\Sigma}|^{1/2}} \exp \left\{ -\frac{1}{2}(\mathbf{x}_n - \boldsymbol{\mu}_k)^T \mathbf{\Sigma}^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k) \right\} \quad (6)$$

and finding $p(\mathcal{C}_k)$, the prior for $\mathcal{C}_k$ class, separately. This route can be computationally expensive, considering we have to invert the covariance matrix, $\mathbf{\Sigma}$.

For general case of K cases with

$$a_k = \mathbf{w}^T \mathbf{x}_n + w_{k0}, \quad (7)$$

we can write the following

$$\mathbf{w}_k = \mathbf{\Sigma}^{-1} \boldsymbol{\mu}_k \quad (8)$$

$$w_{k0} = -\frac{1}{2} \boldsymbol{\mu}_k^T \mathbf{\Sigma}^{-1} \boldsymbol{\mu}_k + \ln p(\mathcal{C}_k). \quad (9)$$

From this, we can conclude two things:

1. If we are using a shared $\mathbf{\Sigma}$ for each class, then $\mathbf{\Sigma}$ can be factored out in (5) leading to decision boundaries to be linear.
2. If we are using $\mathbf{\Sigma}_k$ for each class, this leads to non-linear decision boundaries.

This is illustrated in figure (1).

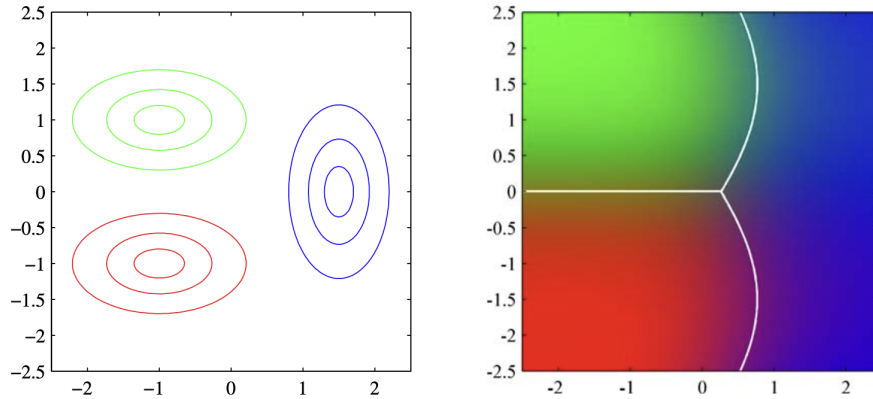


Figure 4.11 The left-hand plot shows the class-conditional densities for three classes each having a Gaussian distribution, coloured red, green, and blue, in which the red and green classes have the same covariance matrix. The right-hand plot shows the corresponding posterior probabilities, in which the RGB colour vector represents the posterior probabilities for the respective three classes. The decision boundaries are also shown. Notice that the boundary between the red and green classes, which have the same covariance matrix, is linear, whereas those between the other pairs of classes are quadratic.

Figure 1: Figure 4.11 from Bishop (2007).

1.4 Softmax regression

Let's have a look at this equation again:

$$p(\mathcal{C}_k|\mathbf{x}_n) = y_k(\mathbf{x}_n, \mathbf{w}) = \frac{\exp(a_k)}{\sum_j^M \exp(a_j)}. \quad (10)$$

If we use $(\mathbf{w}_k^T \mathbf{x}_n + w_{k0})$ to approximate $a_k = \ln(p(\mathbf{x}_n|\mathcal{C}_k)p(\mathcal{C}_k))$ **directly**, such that if $y_k(\mathbf{x}_n) > y_j(\mathbf{x}_n)$ for all $j \neq k$, we assign y_k to $\mathcal{C}_k$. If $y_k(\mathbf{x}_n) = y_j(\mathbf{x}_n)$, the data point $\mathbf{x}_n$ will lie on the decision boundary between $\mathcal{C}_j$ and $\mathcal{C}_k$ classes.

This can be summarized in a matrix form as follows:

$$\begin{bmatrix} p(\mathcal{C}_1|\mathbf{x}; \mathbf{w}_1) \\ p(\mathcal{C}_2|\mathbf{x}; \mathbf{w}_2) \\ \vdots \\ p(\mathcal{C}_K|\mathbf{x}; \mathbf{w}_K) \end{bmatrix} = \begin{bmatrix} \frac{\exp(a_1)}{\sum_j \exp(a_j)} \\ \frac{\exp(a_2)}{\sum_j \exp(a_j)} \\ \vdots \\ \frac{\exp(a_K)}{\sum_j \exp(a_j)} \end{bmatrix} = \begin{bmatrix} \frac{\exp(\mathbf{w}_1^T \mathbf{x} + w_{10})}{\sum_j \exp(\mathbf{w}_j^T \mathbf{x} + w_{j0})} \\ \frac{\exp(\mathbf{w}_2^T \mathbf{x} + w_{20})}{\sum_j \exp(\mathbf{w}_j^T \mathbf{x} + w_{j0})} \\ \vdots \\ \frac{\exp(\mathbf{w}_K^T \mathbf{x} + w_{K0})}{\sum_j \exp(\mathbf{w}_j^T \mathbf{x} + w_{j0})} \end{bmatrix}. \quad (11)$$

What if we find the weights of $y_k(\mathbf{x}_n) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$, where $a_k = \mathbf{w}_k^T \mathbf{x} + w_{k0}$, using a multiclass logistic regression?

This can be done by constructing an error function, and by minimizing the loss iteratively using the training examples. From now on, for notational convenience, we will use $\mathbf{w}_k = (w_{k0}, \mathbf{w}_k)$ combined.

To proceed with this setup, we need to figure out the following things:

1. We need to find $\frac{\partial y_k}{\partial a_j}$ which will be useful later in step 3.
2. We need to construct a likelihood function using the $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and $\mathbf{T} = \{\mathbf{t}_1, \dots, \mathbf{t}_N\}$.
3. Derive a cost/loss function, $E(\mathbf{w}_1, \dots, \mathbf{w}_K)$ by taking the negative log-likelihood of the likelihood function, and find the derivative, $\frac{\partial E}{\partial \mathbf{w}_k}$.
4. Use the gradient descent using $\frac{\partial E}{\partial \mathbf{w}_k}$ to find the optimal value, $\mathbf{w}_k^*$, for all $k = 1, \dots, K$.

1.4.1 Finding $\frac{\partial y_k}{\partial a_j}$.

Note that $y_k(\mathbf{x}) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$ is a function of $a_1, a_2, \dots, a_K$. Therefore, while taking the derivative of y_k w.r.t. a_j , we will have two cases to consider:

1. Taking derivative of y_k w.r.t. a_k .
2. Taking derivative of y_k w.r.t. a_j .

Alternatively, one case is $j = k$, and another is $j \neq k$. When $j = k$,

$$\begin{aligned}
 \frac{\partial y_k}{\partial a_k} &= \frac{\partial}{\partial a_k} \left(\frac{\exp(a_k)}{\sum_j \exp(a_j)} \right) \\
 &= \frac{\exp(a_k) \sum_j \exp(a_j) - \exp(a_k) \exp(a_k)}{\left(\sum_j \exp(a_j) \right)^2} \\
 &= \frac{\exp(a_k)}{\sum_j \exp(a_j)} - \frac{\exp(a_k) \exp(a_k)}{\left(\sum_j \exp(a_j) \right)^2} \\
 &= y_k - y_k^2 \\
 &= y_k(1 - y_k)
 \end{aligned}$$

Similarly, when $j \neq k$,

$$\begin{aligned}
 \frac{\partial y_k}{\partial a_j} &= \frac{\partial}{\partial a_j} \left(\frac{\exp(a_k)}{\sum_j \exp(a_j)} \right) \\
 &= \exp(a_k) \frac{\partial}{\partial a_j} \left(\frac{1}{\sum_j \exp(a_j)} \right) \\
 &= -\exp(a_k) \left(\frac{1}{\left(\sum_j \exp(a_j) \right)^2} \right) \exp(a_j) \\
 &= -\frac{\exp(a_k) \exp(a_j)}{\sum_j \exp(a_j) \sum_j \exp(a_j)} \\
 &= -y_k y_j
 \end{aligned}$$

Combining them together, we get the following result.

$$\frac{\partial y_k}{\partial a_j} = \begin{cases} y_k(1 - y_k) & j = k, \\ -y_k y_j & j \neq k. \end{cases} \quad (12)$$

This can be further combined using an indicator variable, $\delta(k, j)$, defined as:

$$\delta(k, j) = \begin{cases} 1 & j = k, \\ 0 & j \neq k. \end{cases} \quad (13)$$

With $\delta(k, j)$, Eq. (12) can be written as:

$$\frac{\partial y_k}{\partial a_j} = y_k(\delta(k, j) - y_j) = \delta(k, j)y_k - y_k y_j. \quad (14)$$

1.4.2 Constructing likelihood function

For a single data point, $(\mathbf{x}_n, \mathbf{t}_n)$, $p(\mathbf{t}_n|\mathbf{x}_n, \mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K)$ can be written as:

$$p(\mathbf{t}_n|\mathbf{x}_n, \mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{k=1}^K p(\mathcal{C}_k|\mathbf{x}_n)^{t_{nk}}, \quad (15)$$

For instance, if $K = 5$, and $\mathbf{t}_n = (0, 0, 1, 0, 0)$, then, $t_{n3} = 1$, and $t_{n1} = t_{n2} = t_{n4} = t_{n5} = 0$, and

$$p(\mathbf{t}_n|\mathbf{x}_n, \mathbf{w}_1, \dots, \mathbf{w}_K) = p(\mathcal{C}_1|\mathbf{x}_n)^0 \cdot p(\mathcal{C}_2|\mathbf{x}_n)^0 \cdot p(\mathcal{C}_3|\mathbf{x}_n)^1 \cdot p(\mathcal{C}_4|\mathbf{x}_n)^0 \cdot p(\mathcal{C}_5|\mathbf{x}_n)^0 = \prod_{k=1}^5 p(\mathcal{C}_k|\mathbf{x}_n)^{t_{nk}}. \quad (16)$$

Therefore, given $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and $\mathbf{T} = \{\mathbf{t}_1, \dots, \mathbf{t}_N\}$, which are **independent** and **identical**:

$$p(\mathbf{T}|\mathbf{X}, \mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K p(\mathcal{C}_k|\mathbf{x}_n)^{t_{nk}} = \prod_{n=1}^N \prod_{k=1}^K y_{nk}^{t_{nk}}, \quad (17)$$

where $y_{nk} = y_k(\mathbf{x}_n, \mathbf{w}_1, \dots, \mathbf{w}_K) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$.

Putting everything together, the likelihood function will be

$$p(\mathbf{T}|\mathbf{X}, \mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K y_{nk}^{t_{nk}} = \prod_{n=1}^N \prod_{k=1}^K \left(\frac{\exp(a_k)}{\sum_j \exp(a_j)} \right)^{t_{nk}}, \quad (18)$$

with $a_k = \mathbf{w}_k^T \mathbf{x}_n$.

1.4.3 Deriving the loss/cost function $E(\mathbf{w}_1, \dots, \mathbf{w}_K)$

Taking the negative logarithm of (18), we can define a loss/cost function for this multiclass logistic regression.

$$E(\mathbf{w}_1, \dots, \mathbf{w}_K) = -\ln p(\mathbf{T}|\mathbf{X}, \mathbf{w}_1, \dots, \mathbf{w}_K) = -\sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}. \quad (19)$$

This is known as the *cross-entropy* error function for the multiclass classification problem. Now that we have an error function, $E(\mathbf{w}_1, \dots, \mathbf{w}_K)$, we take the derivative of E w.r.t. $\mathbf{w}_j$.

$$\frac{\partial E}{\partial \mathbf{w}_j} = \frac{\partial}{\partial \mathbf{w}_j} \left(-\sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk} \right). \quad (20)$$

Let's evaluate $\frac{\partial t_{nk} \ln y_{nk}}{\partial \mathbf{w}_j}$ first.

$$\begin{aligned}
\frac{\partial t_{nk} \ln y_{nk}}{\partial \mathbf{w}_j} &= \frac{\partial(t_{n,k} \ln y_{n,k}(\mathbf{x}_n))}{\partial y_{n,k}} \cdot \frac{\partial y_{n,k}}{\partial a_j} \cdot \frac{\partial a_j}{\partial \mathbf{w}_j} \\
&= \frac{t_{n,k}}{y_{n,k}} y_{n,k} (\delta(j, k) - y_{n,j}) \mathbf{x}_n \\
&= t_{n,k} (\delta(j, k) - y_{n,j}) \mathbf{x}_n.
\end{aligned}$$

Note that the derivative of a_k w.r.t. $\mathbf{w}_j$ will be

$$\frac{\partial a_j}{\partial \mathbf{w}_j} = \frac{\partial(\mathbf{w}_j^T \mathbf{x}_n)}{\partial \mathbf{w}_j} = \mathbf{x}_n.$$

Now putting $\frac{\partial t_{nk} \ln y_{nk}}{\partial \mathbf{w}_j}$ in Eq. (20), we get

$$\begin{aligned}
\frac{\partial E}{\partial \mathbf{w}_j} &= \frac{\partial}{\partial \mathbf{w}_j} \left(- \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk} \right) \\
&= - \sum_{n=1}^N \sum_{k=1}^K \frac{\partial(t_{nk} \ln y_{nk})}{\partial \mathbf{w}_j} \\
&= - \sum_{n=1}^N \sum_{k=1}^K t_{n,k} (\delta(j, k) - y_{n,j}) \mathbf{x}_n \\
&= \sum_{n=1}^N \sum_{k=1}^K t_{nk} y_{nj} \mathbf{x}_n - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \delta(k, j) \\
&= \sum_{n=1}^N y_{nj} \left(\sum_{k=1}^K t_{nk} \right) \mathbf{x}_n - \sum_{n=1}^N t_{nj} \mathbf{x}_n \\
&= \sum_{n=1}^N (y_{nj} - t_{nj}) \mathbf{x}_n
\end{aligned}$$

Note:

$$\begin{aligned}
\sum_{k=1}^K t_{nk} \delta(k, j) &= t_{n1} \delta(1, j) + t_{n2} \delta(2, j) + \dots + t_{nj} \delta(j, j) + \dots + t_{nK} \delta(K, j) \\
&= t_{n1} \cdot 0 + t_{n2} \cdot 0 + \dots + t_{nj} \delta(j, j) + \dots + t_{nK} \cdot 0 \\
&= t_{nj}.
\end{aligned}$$

1.4.4 Gradient descent using $\frac{\partial E}{\partial \mathbf{w}_j}$.

Each set of weights associated with each class $\mathcal{C}_k$ can be updated sequentially using the simple weight update rule:

$$\mathbf{w}_j^{\tau+1} := \mathbf{w}_j^{\tau} - \eta \frac{\partial E(\mathbf{w}_1^{\tau}, \dots, \mathbf{w}_K^{\tau})}{\partial \mathbf{w}_j^{\tau}} \quad (21)$$

$$:= \mathbf{w}_j^{\tau} - \eta \sum_{n=1}^N (y_{nj} - t_{nj}) \mathbf{x}_n. \quad (22)$$

1.5 Softmax function in neural networks

In a multiclass logistic regression using softmax function, we are essentially mapping the input vectors to predict what class the input vector might likely belong to. As a function, f , a softmax function is $f : \mathbf{x} \rightarrow \mathbf{y}; \mathbf{x} \in \mathbb{R}^D, \mathbf{y} \in \mathbb{R}^K$. Here $\mathbf{y}$ is a vector with $y_1, y_2, \dots, y_K$ values. Here $\sum_j y_j = 1$ because y_k is given by

$$y_k(\mathbf{x}) = \frac{\exp(\mathbf{w}_k^T \mathbf{x})}{\sum_j \exp(\mathbf{w}_j^T \mathbf{x})}. \quad (23)$$

If there is a data transformation involved using a fixed-basis function, $\phi(\mathbf{x}_n)$, then this function is written as:

$$y_k(\mathbf{x}) = \frac{\exp(\mathbf{w}_k^T \phi(\mathbf{x}_n))}{\sum_j \exp(\mathbf{w}_j^T \phi(\mathbf{x}_n))}. \quad (24)$$

In a neural network, the input vector will at least has to be mapped to a hidden layer before its finally mapped to the output layer. This is illustrated in figure 2 with one hidden layer.

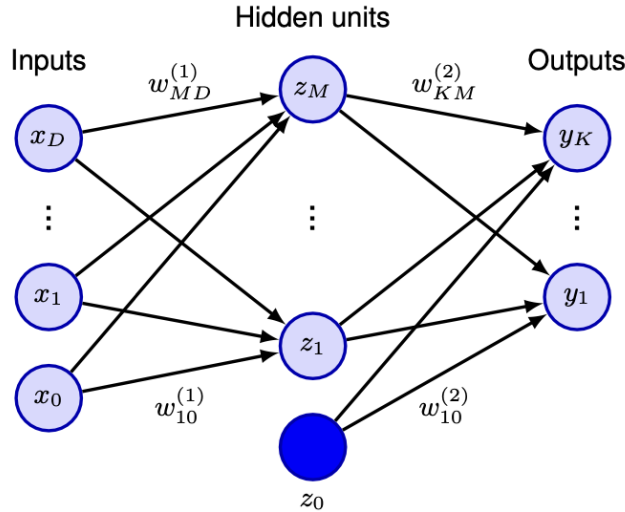


Figure 2: Here, we have D -dimensional input vector, which is mapped to the hidden units $(z_1, z_2, \dots, z_M)$. Those M hidden units are further mapped to K units, $(y_1, y_2, \dots, y_K)$, in the output layer.

As a function, a neural network with one hidden layer will involve two mappings.

1. input layer $\rightarrow$ hidden layer
2. hidden layer $\rightarrow$ output layer

1.5.1 Mapping from input layer to the hidden layer

Let's look at the mapping from the input layer to the hidden layer. With M hidden units, we will need M set of weights to map the input vector, $\mathbf{x}$ to M hidden units. For a j -th hidden unit denoted by z_j , we can write the map as:

$$a_j = w_{j0}^{(1)} + w_{j1}^{(1)}x_1 + w_{j2}^{(1)}x_2 + \dots + w_{jD}^{(1)}x_D = \sum_{i=0}^D w_{ji}^{(1)}x_i \quad (25)$$

$$z_j = h(a_j) = h\left(\sum_{i=0}^D w_{ji}^{(1)}x_i\right). \quad (26)$$

While mapping the neurons/units from preceding layer to the succeeding one, we use an [activation function](#). In Eq. (26), a_j is called *activation*, and $h(\cdot)$ is the activation function. The commonly used activation functions are:

- Hyperbolic tangent: $\tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)}$
- Sigmoid function: $\sigma(a) = \frac{1}{1 + \exp(-a)}$
- Rectified linear unit: $\text{ReLU}(a) = \max(0, a)$

You can find other activation functions [here](#). These activation functions primarily need to be *nonlinear* and *differentiable*. These activation functions are responsible to introduce non-linearity to this machine learning model. With a non-linear and differentiable activation function between the input layer to the hidden layer makes a neural network a [universal approximator](#).

With M hidden units, we will have $\mathbf{x}$ mapping to $a_1, a_2, \dots, a_M$ and each a_j is 'activated' using $h(\cdot)$ activation function. For simplicity, let's assume that $h(a_j) = \tanh(a_j)$

The mapping from the input layer to the first hidden units can be summarized as:

$$\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_M \end{bmatrix} = \begin{bmatrix} h(a_1) \\ h(a_2) \\ \vdots \\ h(a_M) \end{bmatrix} = \begin{bmatrix} \tanh(\sum_{i=0}^D w_{1i}^{(1)}x_i) \\ \tanh(\sum_{i=0}^D w_{2i}^{(1)}x_i) \\ \vdots \\ \tanh(\sum_{i=0}^D w_{Mi}^{(1)}x_i) \end{bmatrix} \quad (27)$$

We are using the superscript (1) to indicate the weights are for the first hidden layer in the neural network. An alternative way to look at this mapping is to think of it as a data transformation, where we are mapping the original input space into a different feature space by using a non-linear activation function.

1.5.2 Mapping from a hidden layer to its subsequent layer

If we are designing a multilayer neural network, we can repeat the same process again, where we assume the first hidden layer as an input signals/vectors for the subsequent layer. Now, $\mathbf{z} = (z_1, z_2, \dots, z_M)^T$ will be mapped to P number of neurons. This can be written in a way similar to Eq. (27) as follows:

$$\begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_P \end{bmatrix} = \begin{bmatrix} h(a_1) \\ h(a_2) \\ \vdots \\ h(a_P) \end{bmatrix} = \begin{bmatrix} \tanh(\sum_{j=0}^M w_{1j}^{(2)} z_j) \\ \tanh(\sum_{j=0}^M w_{2j}^{(2)} z_j) \\ \vdots \\ \tanh(\sum_{j=0}^M w_{Pj}^{(2)} z_j) \end{bmatrix}. \quad (28)$$

1.5.3 Mapping from the penultimate layer to the output layer

The mapping from the penultimate layer to the output layer will have a specific kind of activation function based on the regression or the classification problem.

Regression problem

For a regression problem, we simply map the penultimate layer to the last layer with an identity activation function, which basically means, we simply map to the last layer with the raw output coming out of the penultimate layer. If we have just **one** hidden layer in the neural network with M hidden units, the output layer will have the following values:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix} = \begin{bmatrix} \sum_{j=0}^M w_{1j}^{(2)} z_j \\ \sum_{j=0}^M w_{2j}^{(2)} z_j \\ \vdots \\ \sum_{j=0}^M w_{Kj}^{(2)} z_j \end{bmatrix}. \quad (29)$$

More generically, if there are multiple layers in the neural network, the mapping between the penultimate layer and the output layer can be expressed as:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix} = \begin{bmatrix} \sum_{j=0}^{M'} w_{1j}^{(\ell)} z_j' \\ \sum_{j=0}^{M'} w_{2j}^{(\ell)} z_j' \\ \vdots \\ \sum_{j=0}^{M'} w_{Kj}^{(\ell)} z_j' \end{bmatrix}. \quad (30)$$

where the penultimate layer has M' hidden units, and $\ell - 1$ denoting the number of hidden layers between the input layer to the output layer.

Classification problem

If it's a classification problem with K classes, we specifically use [softmax](#) function as an activation function. This helps us interpret the map between the penultimate layer the

output layer as a probabilistic model. With one hidden layer, the map between the hidden layer with M units to K units in the output layer can be written as:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix} = \begin{bmatrix} h(a_1) \\ h(a_2) \\ \vdots \\ h(a_K) \end{bmatrix} = \begin{bmatrix} \frac{\exp(\sum_{j=0}^M w_{1j}^{(2)} z_j)}{\sum_{k=1}^K \exp(a_k)} \\ \frac{\exp(\sum_{j=0}^M w_{2j}^{(2)} z_j)}{\sum_{k=1}^K \exp(a_k)} \\ \vdots \\ \frac{\exp(\sum_{j=0}^M w_{Kj}^{(2)} z_j)}{\sum_{k=1}^K \exp(a_k)} \end{bmatrix}. \quad (31)$$

Here, $a_k = \sum_{j=0}^M w_{kj}^{(2)} z_j$ and $h(a_k) = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}$.

This can further be written as:

$$\begin{bmatrix} p(C_1 | \mathbf{x}; \mathbf{w}_1 \dots, \mathbf{w}_K) \\ p(C_2 | \mathbf{x}; \mathbf{w}_1 \dots, \mathbf{w}_K) \\ \vdots \\ p(C_K | \mathbf{x}; \mathbf{w}_1 \dots, \mathbf{w}_K) \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix} = \begin{bmatrix} \frac{\exp(\sum_{j=0}^M w_{1j}^{(2)} z_j)}{\sum_{k=1}^K \exp(\sum_{j=0}^M w_{kj}^{(2)} z_j)} \\ \frac{\exp(\sum_{j=0}^M w_{2j}^{(2)} z_j)}{\sum_{k=1}^K \exp(\sum_{j=0}^M w_{kj}^{(2)} z_j)} \\ \vdots \\ \frac{\exp(\sum_{j=0}^M w_{Kj}^{(2)} z_j)}{\sum_{k=1}^K \exp(\sum_{j=0}^M w_{kj}^{(2)} z_j)} \end{bmatrix}. \quad (32)$$

1.6 Loss/Cost function in neural networks

1.6.1 Regression problem in neural networks

With a regression problem, the output value can either be a single value $t \in \mathbb{R}$ or $\mathbf{t} \in \mathbb{R}^K$. In either case, we can define a loss/cost function for a regression problem in neural networks as we did earlier in ‘Linear Models for Regression’. We can define the loss function as:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{t}_n\|^2, \quad (33)$$

however, we must be aware of the fact that $\mathbf{y}(\mathbf{x}_n, \mathbf{w})$ is NO longer a linear model! In case of a single hidden layer neural network, $\mathbf{y}(\mathbf{x}_n, \mathbf{w})$ will be:

$$\mathbf{y}_k(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^M w_{kj}^{(2)} h \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right), \quad (34)$$

where $\mathbf{x}$ is a D -dimensional input vector, mapped to M hidden units in the hidden layer, which is further mapped to K neurons in the output layer.

1.6.2 Classification problem

For a classification problem, we use a *cross-entropy* error function. In case of a classification problem, with a probabilistic approach, we aim to predict the posterior probability $p(\mathcal{C}_k|\mathbf{x})$ for $k = 1, \dots, K$ with a K -class classification. In the neural network, this is achieved by using a softmax function as an activation function in the between the last two layers as shown in Eq. 32. To assess if each $p(\mathcal{C}_k|\mathbf{x})$ is predicting in the right direction, we use the following error function:

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln \underbrace{y_k(\mathbf{x}_n, \mathbf{w})}_{\text{non-linear model}} . \quad (35)$$

This can be derived using the maximum likelihood estimation method, used in Section 1.4.2 above. Although Eq. (35) looks the same as $y_k(\mathbf{x}_n, \mathbf{w})$ used in multi-class logistic regression's error function in Eq. (19), $y_k(\mathbf{x}_n, \mathbf{w})$ is NOT a linear model for classification. Particularly because of the non-linear activation function employed in every layer leading to the penultimate layer. It is a function of several weights stretched over multiple layers in the neural network.

Now that we have error functions for both the regression and the classification problems, how can we find the optimal weights such that $E(\mathbf{w}^*)$ is as low as possible?

1.7 Gradient descent optimization

A simplest approach to using gradient information is to choose the weight update using small steps in the negative direction of $\nabla E(\mathbf{w})$ so that

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)}), \quad (36)$$

where $\eta > 0$, which is the *learning rate*. Unlike linear models for regression and classification, where $E(\mathbf{w})$ was a convex function, $E(\mathbf{w})$ in neural networks may have several local minima. With a batch gradient descent, the gradient will vanish the moment we will encounter any local minima, leading to $\nabla E(\mathbf{w}) = 0$.

In order to avoid this, we can go with online gradient descent, also known as sequential gradient descent or stochastic gradient descent, where we make weight updates based on one data point at a time so that

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)}). \quad (37)$$

An online gradient descent may escape local minima, since a stationary point with respect to the error function for the whole data set will generally not be a stationary point for each data point individually.